

Digital Logic & ALU Design

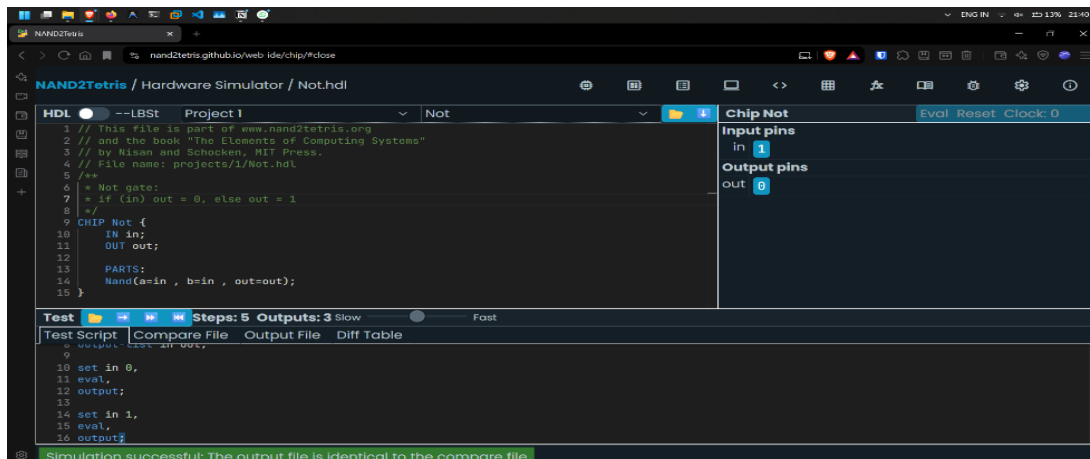
Project1

NOT Gate

Code

```
CHIP Not {  
    IN in;  
    OUT out;  
  
    PARTS:  
    Nand(a=in, b=in, out=out);  
}
```

Screenshot

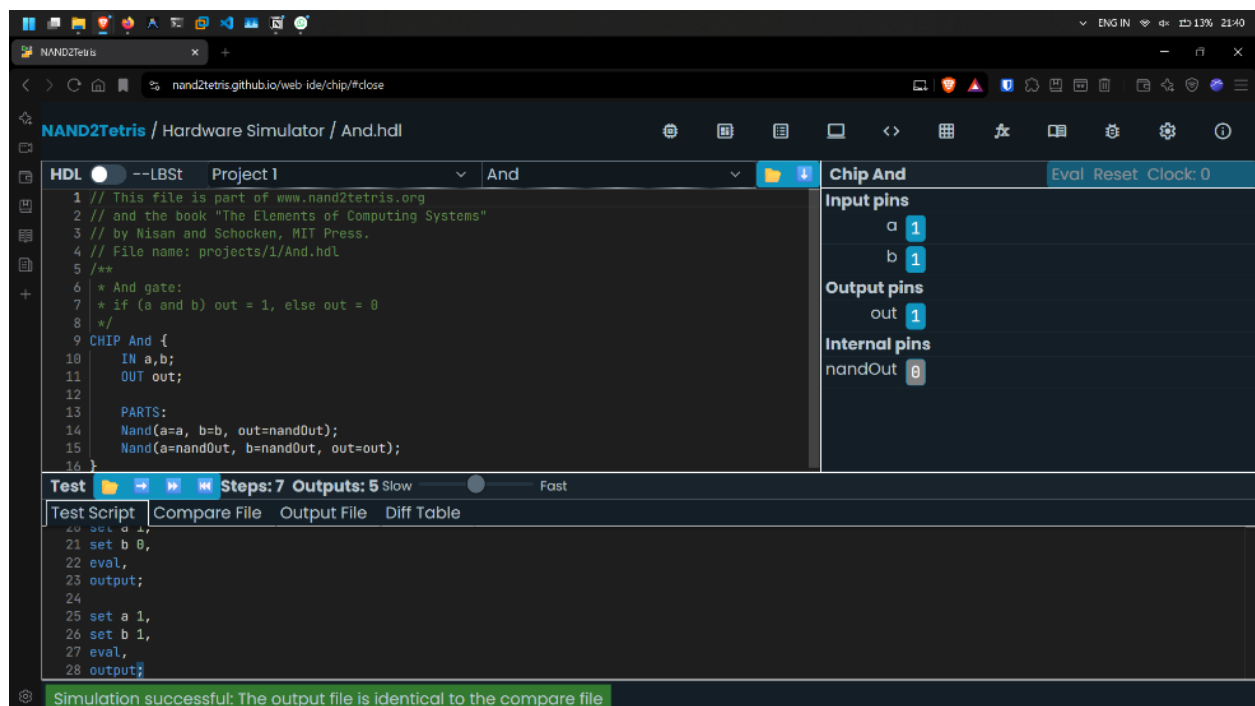


AND Gate

Code

```
CHIP And {  
    IN a, b;  
    OUT out;  
  
    PARTS:  
    Nand(a=a, b=b, out=nandOut);  
    Nand(a=nandOut, b=nandOut, out=out);  
}
```

Screenshot

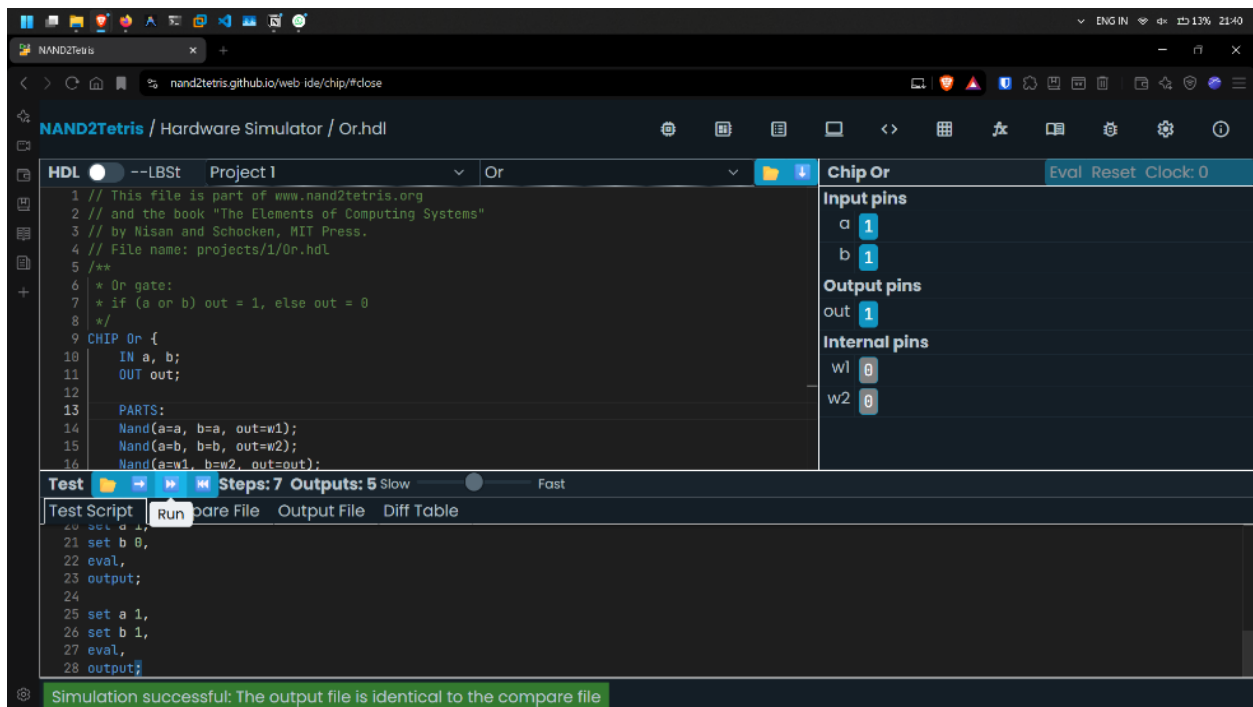


OR Gate

Code

```
CHIP Or {  
    IN a, b;  
    OUT out;  
  
    PARTS:  
    Not(in=a, out=nota);  
    Not(in=b, out=notb);  
    And(a=nota, b=notb, out=andout);  
    Not(in=andout, out=out);  
}
```

Screenshot

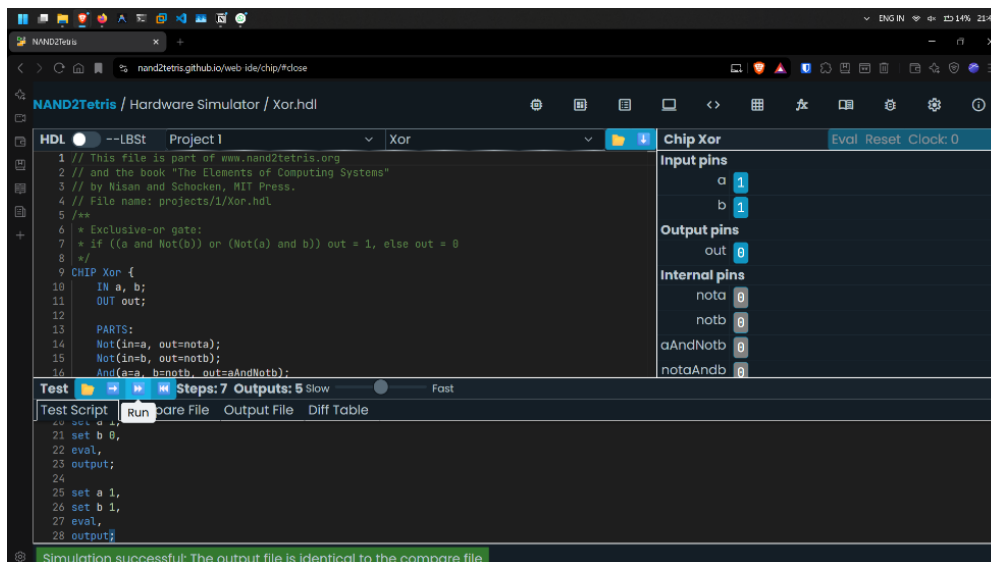


XOR Gate

Code

```
CHIP Xor {
    IN a, b;
    OUT out;
PARTS:
    Or(a=a, b=b, out=orout);
    And(a=a, b=b, out=andout);
    Not(in=andout, out=notand);
    And(a=orout, b=notand, out=out);
}
```

Screenshot



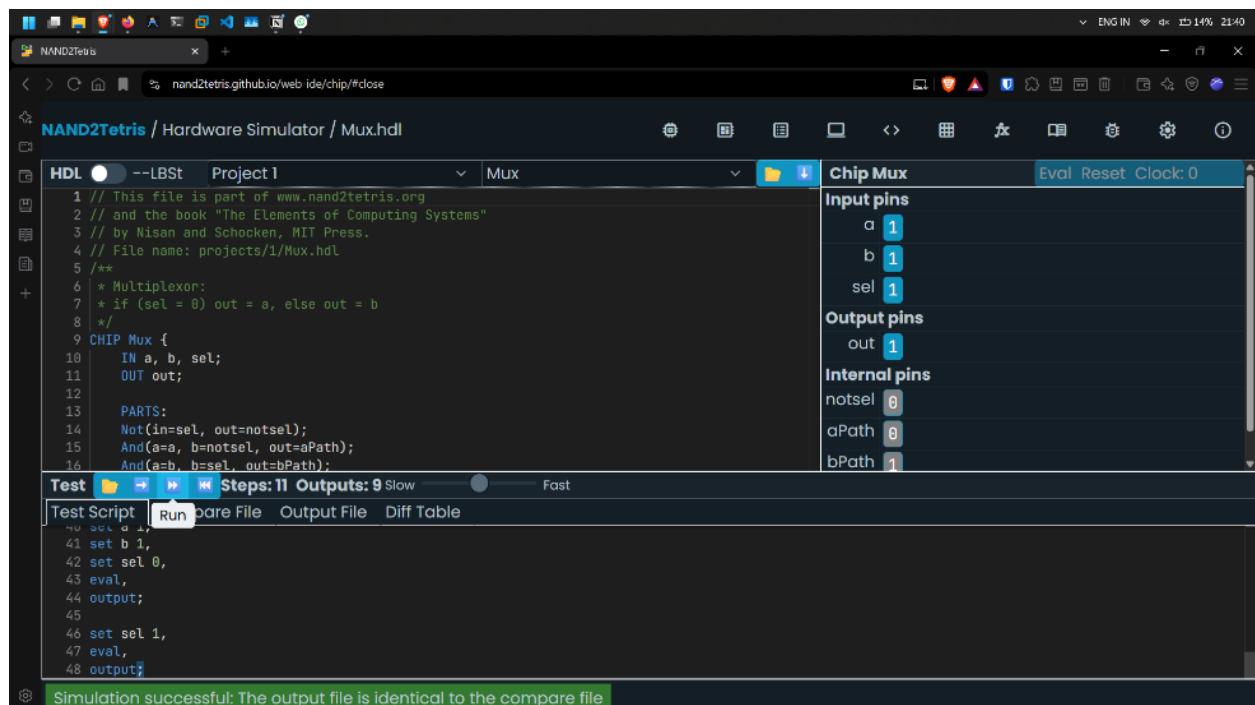
MUX Gate

Code

```
CHIP Mux {
    IN a, b, sel;
    OUT out;

PARTS:
    Not(in=sel, out=notsel);
    And(a=a, b=notsel, out=anda);
    And(a=b, b=sel, out=andb);
    Or(a=anda, b=andb, out=out);
}
```

Screenshot

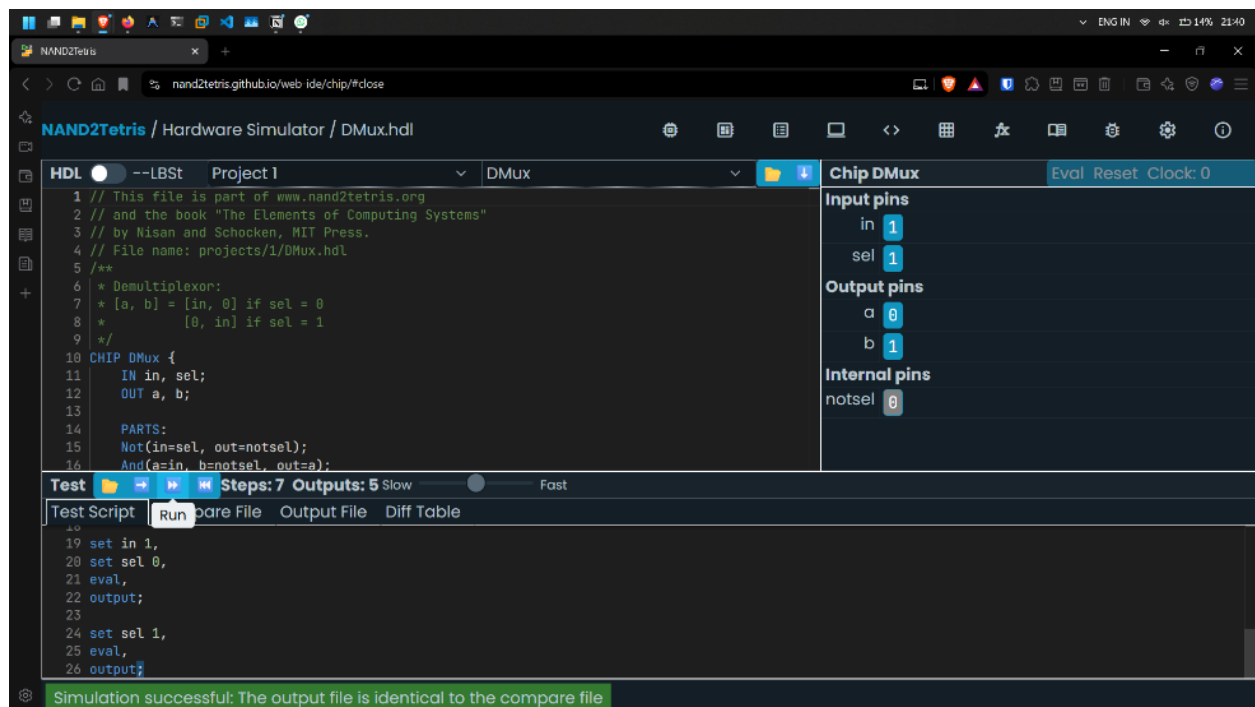


Dmux Gate

Code

```
CHIP DMux {  
    IN in, sel;  
    OUT a, b;  
  
    PARTS:  
        Not(in=sel, out=notsel);  
        And(a=in, b=notsel, out=a);  
        And(a=in, b=sel, out=b);  
}
```

Screenshot



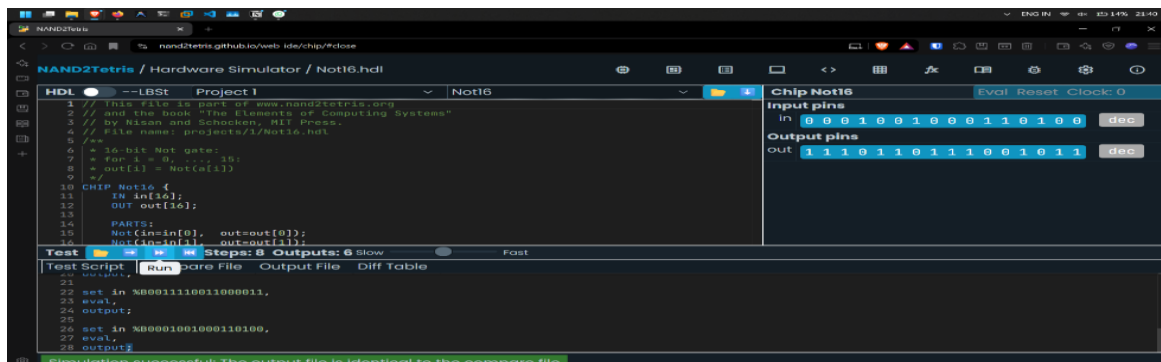
Not 16 Gate

Code

```
CHIP Not16 {
    IN in[16];
    OUT out[16];

    PARTS:
    Not(in=in[0], out=out[0]);
    Not(in=in[1], out=out[1]);
    Not(in=in[2], out=out[2]);
    Not(in=in[3], out=out[3]);
    Not(in=in[4], out=out[4]);
    Not(in=in[5], out=out[5]);
    Not(in=in[6], out=out[6]);
    Not(in=in[7], out=out[7]);
    Not(in=in[8], out=out[8]);
    Not(in=in[9], out=out[9]);
    Not(in=in[10], out=out[10]);
    Not(in=in[11], out=out[11]);
    Not(in=in[12], out=out[12]);
    Not(in=in[13], out=out[13]);
    Not(in=in[14], out=out[14]);
    Not(in=in[15], out=out[15]);
}
```

Screenshot



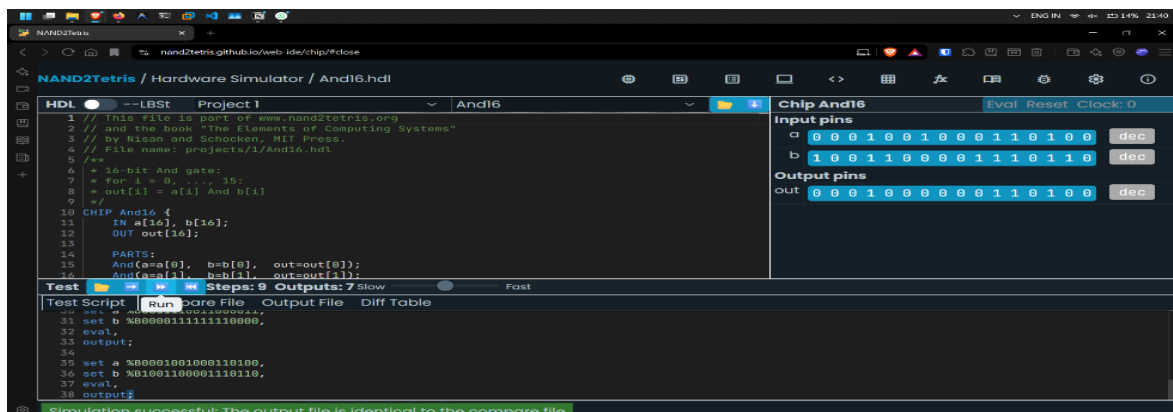
AND16 Gate

Code

```
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    And(a=a[0], b=b[0], out=out[0]);
    And(a=a[1], b=b[1], out=out[1]);
    And(a=a[2], b=b[2], out=out[2]);
    And(a=a[3], b=b[3], out=out[3]);
    And(a=a[4], b=b[4], out=out[4]);
    And(a=a[5], b=b[5], out=out[5]);
    And(a=a[6], b=b[6], out=out[6]);
    And(a=a[7], b=b[7], out=out[7]);
    And(a=a[8], b=b[8], out=out[8]);
    And(a=a[9], b=b[9], out=out[9]);
    And(a=a[10], b=b[10], out=out[10]);
    And(a=a[11], b=b[11], out=out[11]);
    And(a=a[12], b=b[12], out=out[12]);
    And(a=a[13], b=b[13], out=out[13]);
    And(a=a[14], b=b[14], out=out[14]);
    And(a=a[15], b=b[15], out=out[15]);
}
```

Screenshot



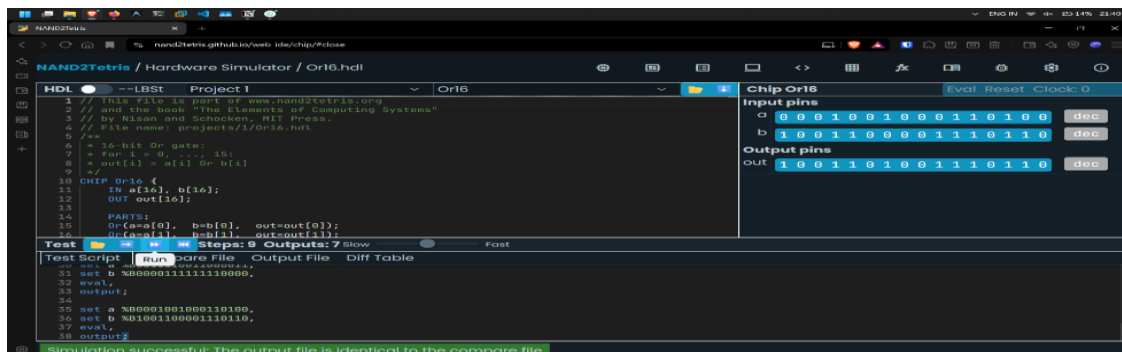
OR16 Gate

Code

```
CHIP Or16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
        Or(a=a[0], b=b[0], out=out[0]);
        Or(a=a[1], b=b[1], out=out[1]);
        Or(a=a[2], b=b[2], out=out[2]);
        Or(a=a[3], b=b[3], out=out[3]);
        Or(a=a[4], b=b[4], out=out[4]);
        Or(a=a[5], b=b[5], out=out[5]);
        Or(a=a[6], b=b[6], out=out[6]);
        Or(a=a[7], b=b[7], out=out[7]);
        Or(a=a[8], b=b[8], out=out[8]);
        Or(a=a[9], b=b[9], out=out[9]);
        Or(a=a[10], b=b[10], out=out[10]);
        Or(a=a[11], b=b[11], out=out[11]);
        Or(a=a[12], b=b[12], out=out[12]);
        Or(a=a[13], b=b[13], out=out[13]);
        Or(a=a[14], b=b[14], out=out[14]);
        Or(a=a[15], b=b[15], out=out[15]);
}
```

Screenshot



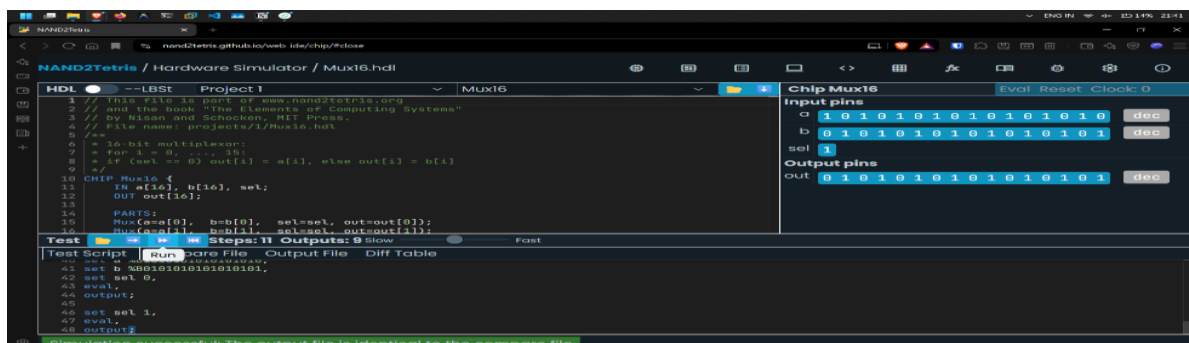
MUX16 Gate

Code

```
CHIP Mux16 {
    IN a[16], b[16], sel;
    OUT out[16];

    PARTS:
        Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
        Mux(a=a[1], b=b[1], sel=sel, out=out[1]);
        Mux(a=a[2], b=b[2], sel=sel, out=out[2]);
        Mux(a=a[3], b=b[3], sel=sel, out=out[3]);
        Mux(a=a[4], b=b[4], sel=sel, out=out[4]);
        Mux(a=a[5], b=b[5], sel=sel, out=out[5]);
        Mux(a=a[6], b=b[6], sel=sel, out=out[6]);
        Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
        Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
        Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
        Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
        Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
        Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
        Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
        Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
        Mux(a=a[15], b=b[15], sel=sel, out=out[15]);
}
```

Screenshot

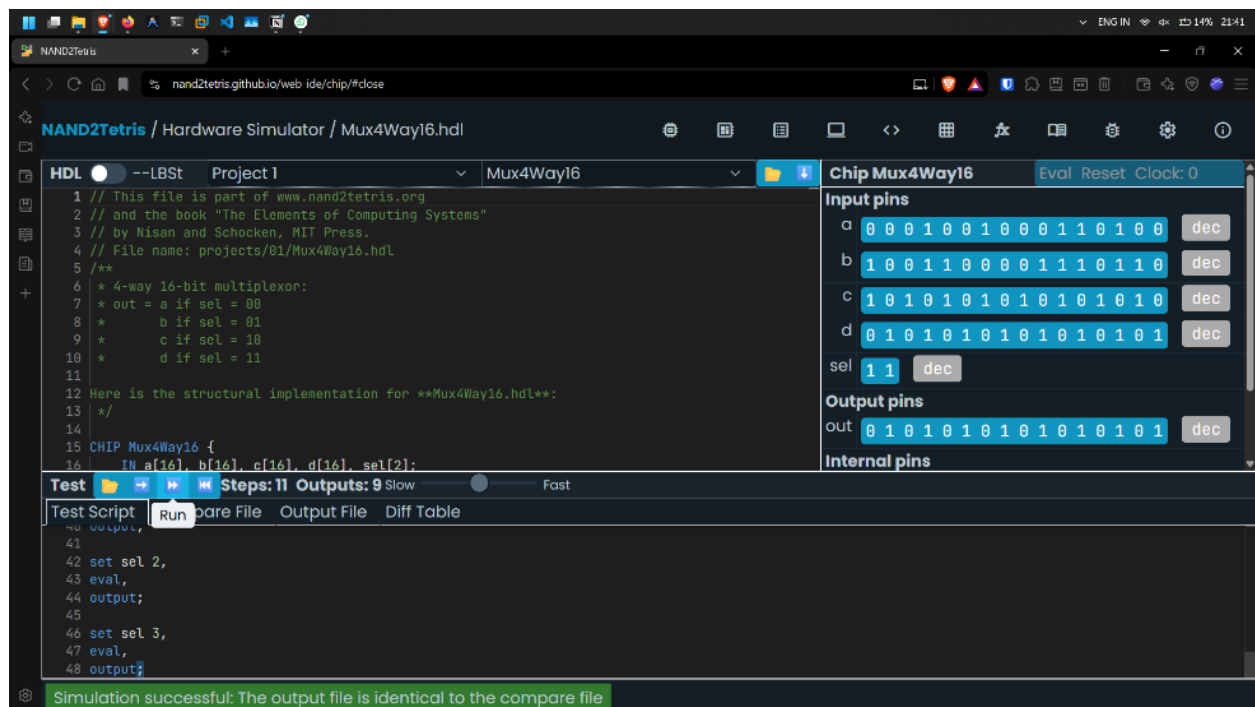


MUX4WAY16 Gate

Code

```
CHIP Mux4Way16 {  
    IN a[16], b[16], c[16], d[16], sel[2];  
    OUT out[16];  
  
    PARTS:  
    Mux16(a=a, b=b, sel=sel[0], out=ab);  
    Mux16(a=c, b=d, sel=sel[0], out=cd);  
    Mux16(a=ab, b=cd, sel=sel[1], out=out);  
}
```

Screenshot

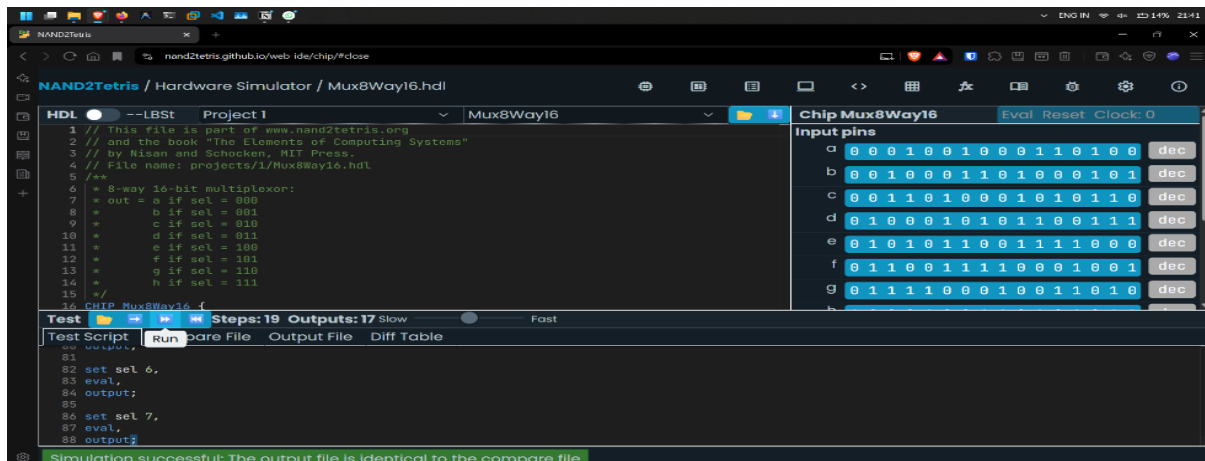


MUX8WAY16 Gate

Code

```
CHIP Mux8Way16 {  
    IN a[16], b[16], c[16], d[16],  
        e[16], f[16], g[16], h[16],  
        sel[3];  
    OUT out[16];  
  
    PARTS:  
        Mux4Way16(a=a, b=b, c=c, d=d,  
            sel=sel[0..1], out=out1);  
  
        Mux4Way16(a=e, b=f, c=g, d=h,  
            sel=sel[0..1], out=out2);  
  
        Mux16(a=out1, b=out2,  
            sel=sel[2], out=out);  
}
```

Screenshot



DMUX4WAY Gate

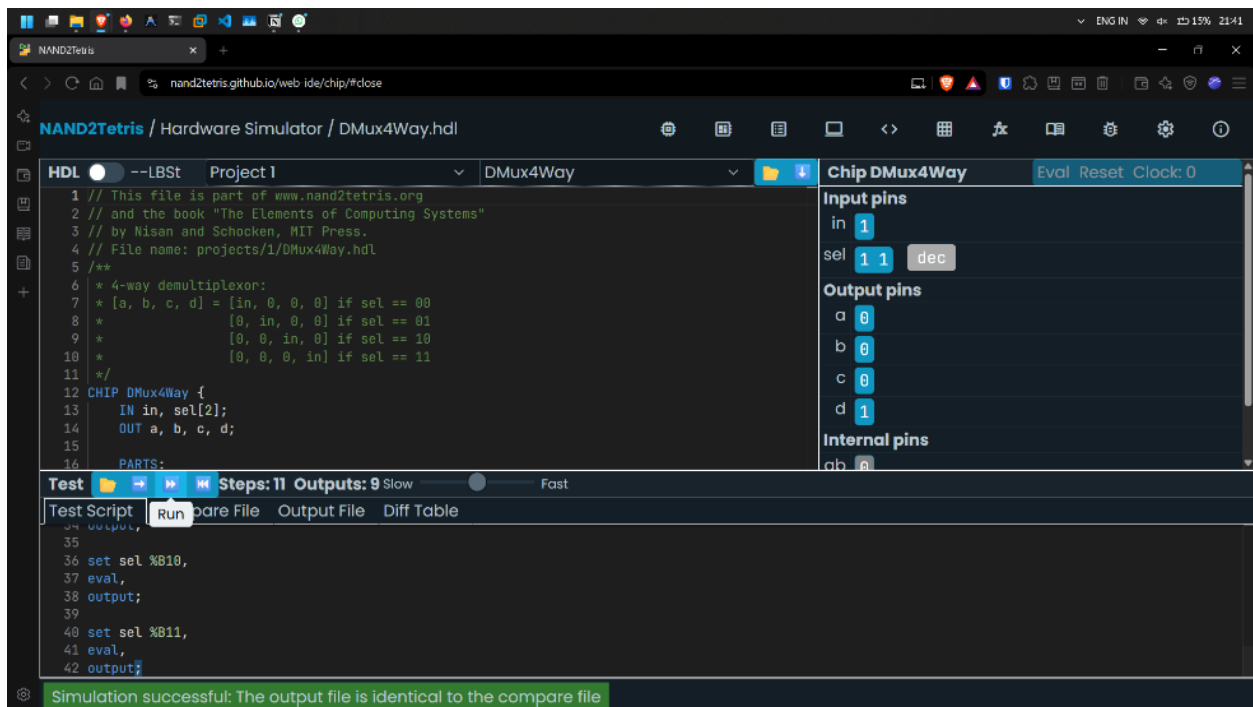
Code

```
CHIP DMux4Way {
    IN in, sel[2];
    OUT a, b, c, d;

    PARTS:
    DMux(in=in, sel=sel[1], a=o1, b=o2);

    DMux(in=o1, sel=sel[0], a=a, b=b);
    DMux(in=o2, sel=sel[0], a=c, b=d);
}
```

Screenshot



DMUX8WAY Gate

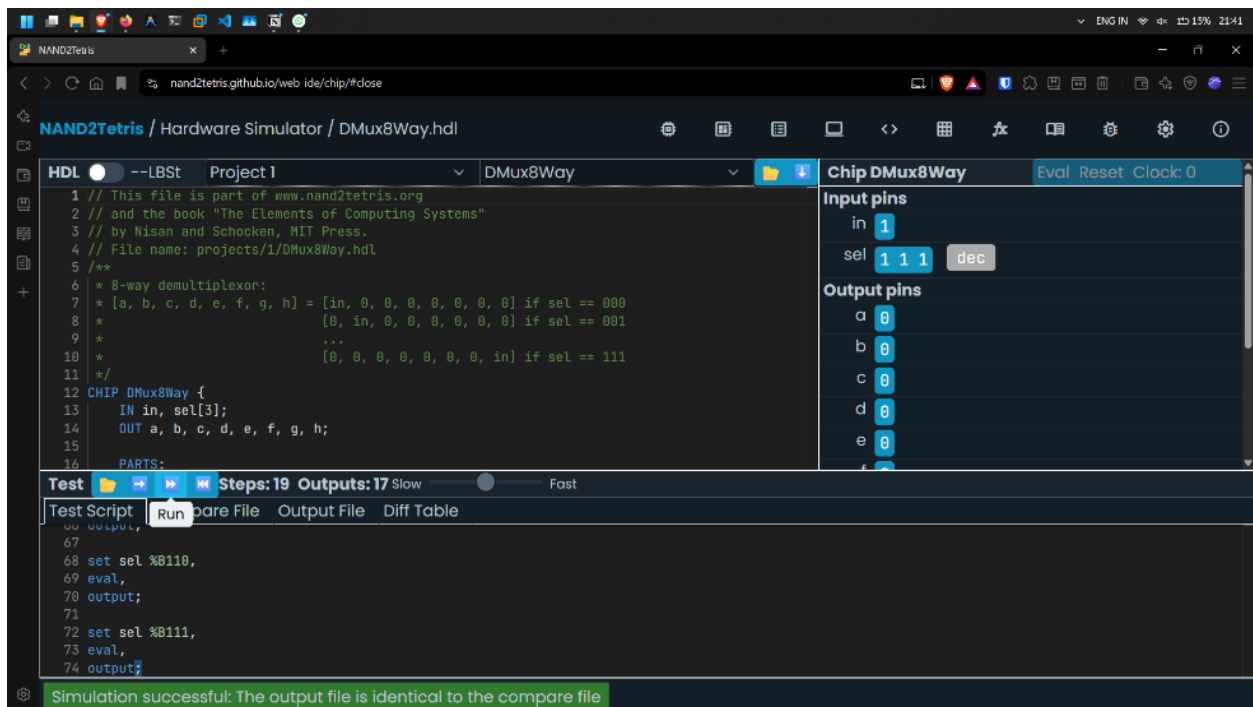
Code

```
CHIP DMux8Way {
    IN in, sel[3];
    OUT a, b, c, d, e, f, g, h;

    PARTS:
    DMux(in=in, sel=sel[2], a=topGroup, b=bottomGroup);

    DMux4Way(in=topGroup, sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=bottomGroup, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

Screenshot



Project2

HalfAdder

Code

```
CHIP HalfAdder {  
    IN a, b;  
    OUT sum,  
        carry;  
  
    PARTS:  
    Xor(a=a, b=b, out=sum);  
    And(a=a, b=b, out=carry);  
}
```

Screenshot

The screenshot displays a digital logic simulator interface. The top panel shows the project name 'Project 2' and the selected chip 'HalfAdder'. The main area is divided into two sections: the left section shows the HDL code for the HalfAdder chip, and the right section shows the chip's input and output pins. The input pins 'a' and 'b' are both set to 1. The output pins 'sum' and 'carry' are shown with values 0 and 1, respectively. The bottom panel shows the test script and the simulation results. The test script includes commands to set inputs and evaluate the outputs. The simulation results show that the simulation was successful, with the output file being identical to the compare file.

```
1 // This file is part of www.nand2tetris.org  
2 // and the book "The Elements of Computing Systems"  
3 // by Nisan and Schocken, MIT Press.  
4 // File name: projects/2/HalfAdder.hdl  
5 /**  
6  * Computes the sum of two bits.  
7  */  
8 CHIP HalfAdder {  
9     IN a, b;      // 1-bit inputs  
10    OUT sum,      // Right bit of a + b  
11        carry;    // Left bit of a + b  
12  
13    PARTS:  
14    Xor(a=a, b=b, out=sum);  
15    And(a=a, b=b, out=carry);  
16 }
```

Test Script: Compare File Output File Diff Table

```
21 set b 0;  
22 eval;  
23 output;  
24  
25 set a 1;  
26 set b 1;  
27 eval;  
28 output;
```

Simulation successful: The output file is identical to the compare file

FullAdder

Code

```
CHIP FullAdder {
    IN a, b, c; // 1-bit inputs
    OUT sum,     // Right bit of a + b + c
        carry;  // Left bit of a + b + c

    PARTS:
    HalfAdder(a=a, b=b, sum=s1, carry=c1);

    HalfAdder(a=s1, b=c, sum=sum, carry=c2);

    Or(a=c1, b=c2, out=carry);
}
```

Screenshot

The screenshot displays a digital logic simulator interface. The top bar shows 'HDL' selected, 'Project 2' as the current project, and 'FullAdder' as the selected chip. The 'Chip FullAdder' panel on the right shows the internal state: 'Input pins' (a=1, b=1, c=1), 'Output pins' (sum=1, carry=1), and 'Internal pins' (s1=0, c1=1). The main editor shows the Verilog code for the FullAdder chip. The bottom panel shows the test script with the following commands: '39 set b 1;', '40 set c 0;', '41 eval;', '42 output;', '43', '44 set c 1;', '45 eval;', '46 output;'. A green status bar at the bottom indicates 'Simulation successful: The output file is identical to the compare file'.

```
5 /**
6  * Computes the sum of three bits.
7  */
8 CHIP FullAdder {
9     IN a, b, c; // 1-bit inputs
10    OUT sum,     // Right bit of a + b + c
11        carry;  // Left bit of a + b + c
12
13    PARTS:
14    HalfAdder(a=a, b=b, sum=s1, carry=c1);
15
16    HalfAdder(a=s1, b=c, sum=sum, carry=c2);
17
18    Or(a=c1, b=c2, out=carry);
19 }
```

Test Script | Compare File | Output File | Diff Table

```
39 set b 1,
40 set c 0,
41 eval,
42 output;
43
44 set c 1,
45 eval,
46 output;
```

Simulation successful: The output file is identical to the compare file

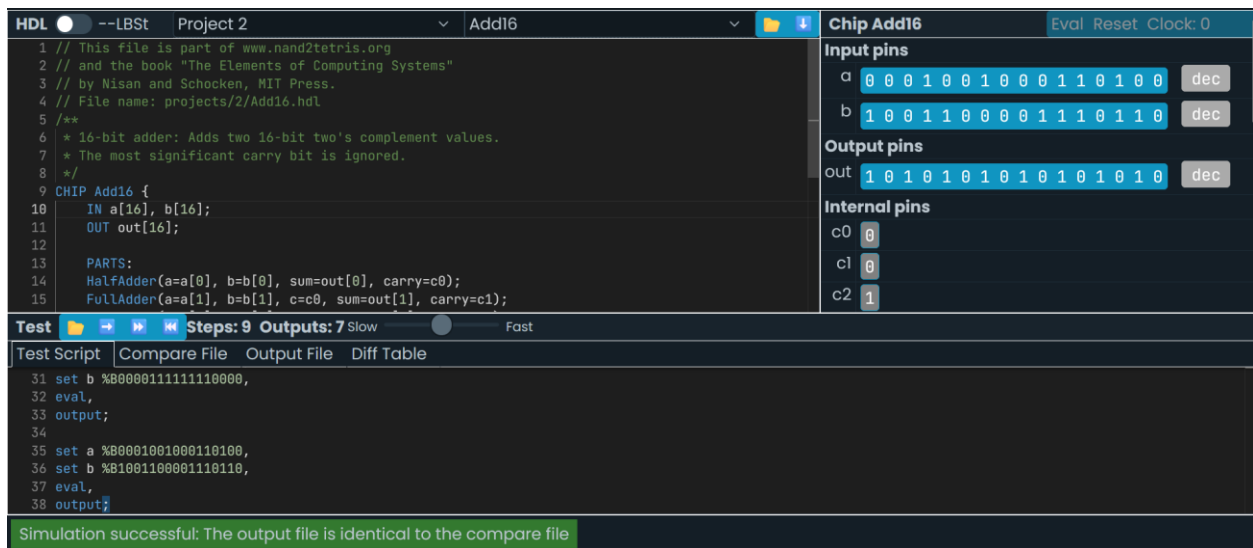
Add16

Code

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);
    FullAdder(a=a[1], b=b[1], c=c0, sum=out[1], carry=c1);
    FullAdder(a=a[2], b=b[2], c=c1, sum=out[2], carry=c2);
    FullAdder(a=a[3], b=b[3], c=c2, sum=out[3], carry=c3);
    FullAdder(a=a[4], b=b[4], c=c3, sum=out[4], carry=c4);
    FullAdder(a=a[5], b=b[5], c=c4, sum=out[5], carry=c5);
    FullAdder(a=a[6], b=b[6], c=c5, sum=out[6], carry=c6);
    FullAdder(a=a[7], b=b[7], c=c6, sum=out[7], carry=c7);
    FullAdder(a=a[8], b=b[8], c=c7, sum=out[8], carry=c8);
    FullAdder(a=a[9], b=b[9], c=c8, sum=out[9], carry=c9);
    FullAdder(a=a[10], b=b[10], c=c9, sum=out[10], carry=c10);
    FullAdder(a=a[11], b=b[11], c=c10, sum=out[11], carry=c11);
    FullAdder(a=a[12], b=b[12], c=c11, sum=out[12], carry=c12);
    FullAdder(a=a[13], b=b[13], c=c12, sum=out[13], carry=c13);
    FullAdder(a=a[14], b=b[14], c=c13, sum=out[14], carry=c14);
    FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);
}
```

Screenshot



Inc16

Code

```
CHIP Inc16 {
    IN in[16];
    OUT out[16];

    PARTS:
        Add16(a=in, b[0]=true, b[1..15]=false, out=out);
}
```

Screenshot

HDL

--LBSt

Project 2

Inc16

Chip Inc16

Eval Reset Clock: 0

```

1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/Inc16.hdl
5 /**
6  * 16-bit incrementer:
7  * out = in + 1
8  */
9 CHIP Inc16 {
10     IN in[16];
11     OUT out[16];
12
13     PARTS:
14         Add16(a=in, b[0]=true, b[1..15]=false, out=out);
15 }

```

Input pins

in 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 dec

Output pins

out 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 dec

Test

Steps: 7 Outputs: 5

Slow Fast

Test Script

Compare File

Output File

Diff Table

```

17
18 set in %B0000000000000101, // in = 5
19 eval,
20 output;
21
22 set in %B1111111111111011, // in = -5
23 eval,
24 output;

```

Simulation successful: The output file is identical to the compare file

ALU

Code

```

CHIP ALU {
    IN
        x[16], y[16], // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f, // compute (out = x + y) or (out = x & y)?
        no; // negate the out output?

    OUT
        out[16], // 16-bit output
        zr, // if (out == 0) equals 1, else 0
        ng; // if (out < 0) equals 1, else 0

    PARTS:

Mux16(a=x, b=false, sel=zx, out=x1);
Not16(in=x1, out=nx1);
Mux16(a=x1, b=nx1, sel=nx, out=x2);

```

```

Mux16(a=y, b=false, sel=zy, out=y1);
Not16(in=y1, out=ny1);
Mux16(a=y1, b=ny1, sel=ny, out=y2);

And16(a=x2, b=y2, out=fAnd);
Add16(a=x2, b=y2, out=fAdd);
Mux16(a=fAnd, b=fAdd, sel=f, out=fOut);

Not16(in=fOut, out=nfOut);
Mux16(a=fOut, b=nfOut, sel=no, out=out, out[0..7]=low8,
out[8..15]=high8, out[15]=ng);

Or8Way(in=low8, out=orLow);
Or8Way(in=high8, out=orHigh);
Or(a=orLow, b=orHigh, out=notZero);
Not(in=notZero, out=zr);

}

```

Screenshot

The screenshot displays a Verilog simulation environment. The top panel shows the HDL code for the ALU chip, which implements various operations like zero, negate, compute, and output. The right panel shows the input pins for the chip, with values for x, y, zx, nx, zy, ny, f, and no. The bottom panel shows the test script, which sets up the inputs and evaluates the output. A status bar at the bottom indicates that the simulation was successful.

Chip ALU Eval Reset Clock: 0

Input pins

Pin	Value	Label
x	00000000000000010001	dec
y	00000000000000000011	dec
zx	0	
nx	1	
zy	0	
ny	1	
f	0	
no	1	

Test Steps: 41 Outputs: 37 Slow Fast

Test Script Compare File Output File Diff Table

```

367 set zx 0,
368 set nx 1,
369 set zy 0,
370 set ny 1,
371 set f 0,
372 set no 1,
373 eval,
374 output;

```

Simulation successful: The output file is identical to the compare file

Submitted by- Yokshit Rajora
(2501410013)